

数据指标提取技术路线

指标可行性深度分析

我们先将指标进行归类。这 10 个指标中，有 7 个可行，3 个不可行（不建议在 Demo 阶段实现）。

一、语义分割类指标（绿视率、蓝视率、人造物占比、天空可视率）

对于这类计算某种元素在画面中占比的指标，最通用、最成熟的方法是使用语义分割（Semantic Segmentation）模型。

- 推荐通用模型：

- **SegFormer** (huggingface 上的 `nvidia/segformer-b0-finetuned-ade20k`)：极其轻量、速度极快，在 CPU/GPU 上均可在 1 秒内完成一张图的推理，支持 ADE20K 协议（原生支持识别天、水、草、树、建筑、道路等 150 种类别）。

- 计算逻辑：

- a. 用 SegFormer 对 2:1 的全景图进行推理，得到每个像素的类别 ID。
- b. 统计特定类别的像素总数，除以全景图的总像素数。

1. 绿视率 【绝对可行 | 开发耗时：1.5 小时】

- **计算逻辑**：统计标签为“树木 (tree)”、“草地 (grass)”、“植物 (plant)”的像素数 / 图像总像素数。
- **代码实现难度**：极低。

2. 蓝视率 (水体可视率) 【绝对可行 | 开发耗时：1.5 小时】

- **计算逻辑**：统计标签为“水 (water)”、“河流 (river)”、“湖泊 (lake)”等水体像素数 / 图像总像素数。
- **代码实现难度**：极低。

3. 天空可视率 【绝对可行 | 开发耗时：1.5 小时】

- **计算逻辑**：统计标签为“天空 (sky)”的像素数 / 图像总像素数。
- **代码实现难度**：极低。

4. 人造物占比 【绝对可行 | 开发耗时：2 小时】

- **计算逻辑**：统计标签为“建筑 (building)”、“道路 (road)”、“人行道 (sidewalk)”、“围墙 (fence)”、“汽车 (car)”等所有人造物像素的总和 / 图像总像素数。
- **代码实现难度**：极低。

二、纯数学/图像算法类指标（色彩丰富度、边缘密度、天际线变化率）

这类指标不需要复杂的 AI 模型，使用 Python 的图像处理库（OpenCV, NumPy）即可直接计算，运算速度在毫秒级。

5. 色彩丰富度 🎨 【绝对可行 | 开发耗时：1.5 小时】

- 推荐工具：OpenCV + NumPy。
- 计算逻辑：
 - a. 将 JPG 图片转换到 HSV 空间。
 - b. 过滤掉饱和度 $S < 0.1$ 和亮度 $V < 0.1$ （剔除黑白灰）。
 - c. 将 H （色相 0~180）划分为 24 个区间。
 - d. 统计落入每个区间像素的占比，当占比 $> 0.5\%$ 时判定为有效颜色。
 - e. 使用表格中的公式计算 $N_{\text{effective}}$ （色彩熵）。
- 代码实现难度：中等（只需 20 行纯 Python 数学计算）。

6. 边缘密度 🖼️ 【绝对可行 | 开发耗时：1 小时】

- 推荐工具：OpenCV (Canny 算子)。
- 计算逻辑：
 - a. 将全景图转为灰度图。
 - b. 使用 `cv2.Canny(gray, 100, 200)` 提取边缘像素。
 - c. 边缘像素总数 / 图像总像素数。
- 代码实现难度：极低（5 行代码）。

7. 天际线变化率 🏙️ 【中等可行 | 开发耗时：3 小时】

- 推荐工具：SegFormer（天空 Mask）+ NumPy。
- 计算逻辑：
 - a. 利用前面分割出的“天空 Mask”（天空区域为 1，其余为 0）。
 - b. 按列（横向 x 轴）寻找每一列中天空的最底部边界（即天际线高度坐标 y_x ）。
 - c. 套用表格中的一阶差分公式： $\frac{\sum |y_{x+1} - y_x|}{(W-1)H} \times 100\%$ 。
- 代码实现难度：中等。

三、🛑 不可行指标（不建议在 1 天内开发）

以下指标在学术上可行，但对于 8-10 小时的敏捷开发来说极度困难，建议在 Demo 阶段予以剔除或简化。

8. 建筑（表观）高度 ❌ 【不可行 | 预计开发耗时：30 小时+】

- 不可行原因：

- 要计算“平均楼层数”，AI 必须能分辨出每一栋独立的建筑（这需要 Instance Segmentation 实例分割，如 Mask2COCO，开发难度剧增）。
- AI 很难在全景畸变图像中准确数出“每一栋楼有几层”，目前没有现成的开箱即用模型能高精度数楼层，需要大量人工规则或多视角投影几何计算。

- **1天内的妥协替代方案：**直接计算“建筑面积占比”（利用 SegFormer 提取建筑像素占比），这通常能间接反映建筑的视觉压迫感和高度。

9. 人造设施量 ❌ 【不可行 | 预计开发耗时：20 小时+】

- 不可行原因：

- 这属于**目标检测（Object Detection）或实例分割**。
- 虽然 YOLOv8 等模型可以检测车辆、路灯、椅子，但全景图（2:1）存在严重的边缘拉伸畸变，标准的 YOLO 模型直接输入会产生大量的漏检和误检。要校正畸变并准确计数，开发和调试成本极高。

- **1天内的妥协替代方案：**用“人造物占比”（指标 4）来代替。

10. 遮阴率 ❌ 【不可行 | 预计开发耗时：24 小时+】

- 不可行原因：

- 表格中提到要识别“地面阴影像素”。阴影的识别在计算机视觉中是一个独立且复杂的难题（Shadow Detection）。
- 阴影的颜色深度、边缘受拍摄当天的天气（多云、晴天）、太阳高度角、路面材质影响极大，使用现成的语义分割模型极难精准剥离出“树木遮挡的阴影”，极易把黑色的沥青路面误判为阴影。

🔧 推荐的 1 天实现技术栈及核心代码

我们过滤掉不可行指标后，保留 **绿视率、蓝视率、天空可视率、人造物占比、色彩丰富度、边缘密度、天际线变化率** 7 个核心指标。

在你的 Notebook 中，只需要 `pip install transformers torch opencv-python-headless`，然后运行以下合并好的极简计算函数：

Python

代码块

```
1 import torch
2 import cv2
3 import numpy as np
```

```

4  from transformers import SegformerImageProcessor,
   SegformerForSemanticSegmentation
5  from PIL import Image
6
7  # 1. 载入极轻量、开箱即用的语义分割模型（首次运行会自动下载约 100MB 模型）
8  processor = SegformerImageProcessor.from_pretrained("nvidia/segformer-b0-
   finetuned-ade20k")
9  model = SegformerForSemanticSegmentation.from_pretrained("nvidia/segformer-b0-
   finetuned-ade20k")
10
11 def calculate_metrics(image_path):"""
12     输入一张 2:1 全景 JPG 图片，一键计算 6 个关键指标
13     """
14     img = cv2.imread(image_path)
15     h, w, _ = img.shape
16     total_pixels = h * w
17
18     # ==== A. 运行语义分割 ====
19     pil_img = Image.open(image_path)
20     inputs = processor(images=pil_img, return_tensors="pt")
21     with torch.no_grad():
22         outputs = model(**inputs)
23
24     # 尺寸恢复至原图大小
25     logits = outputs.logits
26     upsampled_logits = torch.nn.functional.interpolate(
27         logits, size=pil_img.size[::-1], mode="bilinear", align_corners=False
28     )
29     seg_mask = upsampled_logits.argmax(dim=1)[0].numpy()
30
31     # ADE20K 类别标签映射（常见部分）：# 天空=2，建筑=1，道路=6，树木=4，草地=9，陆地/
   泥土=13，水=21，汽车=20，围墙=32
32     sky_mask = (seg_mask == 2)
33     tree_mask = (seg_mask == 4) | (seg_mask == 9) # 树 + 草
34     water_mask = (seg_mask == 21)
35
36     # 人造物（建筑 + 道路 + 汽车 + 栅栏 + 墙）
37     built_mask = np.isin(seg_mask, [1, 6, 20, 32, 11, 43])
38
39     # 2. 计算占比类指标
40     sky_ratio = np.sum(sky_mask) / total_pixels
41     green_ratio = np.sum(tree_mask) / total_pixels
42     blue_ratio = np.sum(water_mask) / total_pixels
43     built_ratio = np.sum(built_mask) / total_pixels
44
45     # ==== B. 纯图像算法：边缘密度（Canny） ====
46     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

```

```

47 blurred = cv2.GaussianBlur(gray, (5, 5), 0)
48 edges = cv2.Canny(blurred, 50, 150)
49 edge_density = np.sum(edges > 0) / total_pixels
50
51 # ==== C. 纯图像算法：色彩丰富度 (HSV 色彩熵) ====
52 hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
53 h_channel = hsv[:, :, 0]
54 s_channel = hsv[:, :, 1]
55 v_channel = hsv[:, :, 2]
56
57 # 排除低饱和度和极暗像素 (黑白灰)
58 valid_color_mask = (s_channel > 25) & (v_channel > 25)
59 valid_h = h_channel[valid_color_mask]
60
61 if len(valid_h) > 0:
62     # 划分为 24 个色相区间
63     counts, _ = np.histogram(valid_h, bins=24, range=(0, 180))
64     probs = counts / len(valid_h)
65     probs = probs[probs > 0] # 排除概率为0的区间# 计算香农熵
66     color_entropy = -np.sum(probs * np.log(probs + 1e-10))
67     # 转换为有效颜色数 (指数熵)
68     effective_color_num = np.exp(color_entropy)
69 else:
70     effective_color_num = 1.0# ==== D. 图像几何：天际线变化率 ====# 寻找每一列
天空(Value=1)的最底部的 y 坐标
71 sky_y_coords = []
72 for col in range(w):
73     sky_indices = np.where(sky_mask[:, col])[0]
74     if len(sky_indices) > 0:
75         sky_y_coords.append(sky_indices[-1]) # 天空最底部的像素行号else:
76         sky_y_coords.append(h) # 如果该列没有天空，默认落到图像最底部
77
78 sky_y_coords = np.array(sky_y_coords)
79 # 计算相邻列的高度绝对差值
80 y_diffs = np.abs(np.diff(sky_y_coords))
81 skyline_variance_rate = np.sum(y_diffs) / ((w - 1) * h)
82
83 # 格式化输出结果return {
84     "绿视率 (Green View)": f"{green_ratio * 100:.2f}%",
85     "蓝视率 (Blue View)": f"{blue_ratio * 100:.2f}%",
86     "天空可视率 (Sky View)": f"{sky_ratio * 100:.2f}%",
87     "人造物占比 (Man-made Ratio)": f"{built_ratio * 100:.2f}%",
88     "边缘密度 (Edge Density)": f"{edge_density * 100:.2f}%",
89     "有效色彩数量 (Color richness)": f"{effective_color_num:.2f}",
90     "天际线变化率 (Skyline Variance)": f"{skyline_variance_rate * 100:.2f}%"
91 }
92

```

```
93 # 测试运行# print(calculate_metrics("ready_for_ai.jpg"))
```

总结

这套方案仅通过引入一个极轻量的 **SegFormer**（无代码门槛，几行代码自动加载），就完成了 4 个最核心的占比指标；再加上 **OpenCV 矩阵运算** 完成了 3 个物理与视觉特征指标。整个技术栈运行在 CPU 上也只需 2~3 秒，完美满足在 **8-10小时** 内开发出一个高性能高完整度 Demo 的需求！